SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	**CSA1205**	**Course Title:**	**Computer Architecture for Multicore Systems**
CO Assessed:	**CO2**	**Assessment:**	**ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK**
Weightage:	**20%**	**Total Marks:**	**25**
CO2 Statement:	*Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)*		
Student Name:	**Kaviyaasri M**	**Reg.No.:**	**192312614**
Department:	**ECE**	**Date:**	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors.

Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	**Debugging Approach & Analysis (20%)**	**Correctness of Debugged Solution (25%)**	**Optimization Techniques Applied (20%)**	**Testing and Documentation (15%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Q1. Signed Integer Addition Using 2's Complement

1. Problem Identification

For signed 8-bit integers, the valid range is:

$-128 \leq N \leq 127$

Given:

$120+50=170$

Since 170 is outside the 8-bit signed range, an overflow occurs.

Binary representation:

120=01111000 50=00110010

Addition:

```
  01111000
+ 00110010
__________
  10101010
__________
```

The result has MSB = 1 and is therefore interpreted as negative in 2's complement. This is an incorrect signed result caused by overflow.

Buggy Code

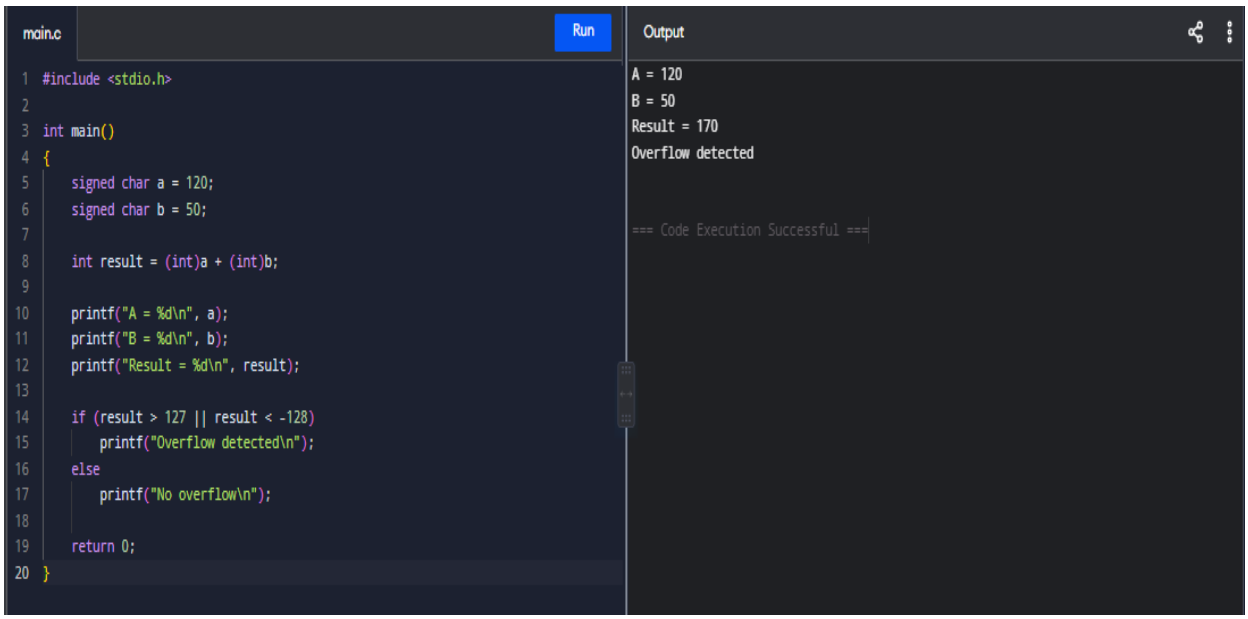
```
#include <stdio.h>

int main()

{

    signed char a = 120;

    signed char b = 50;

    signed char result = a + b;

    printf("Result = %d\n", result);

    return 0;

}
```

Bug Identified

The result of the addition is stored directly in an 8-bit signed variable. The mathematical result 170 cannot be represented in that range.

Debudded Code:

```
#include <stdio.h>

int main(){

    signed char a = 120;

    signed char b = 50;

    int result = (int)a + (int)b;

    printf("A = %d\n", a); printf("B = %d\n", b);

    printf("Result = %d\n", result);

    if (result > 127 || result < -128)

        printf("Overflow detected\n");

    else

        printf("No overflow\n");

    return 0;}
```



Debugging Analysis

The operands are promoted to a wider integer type before addition. The result is then checked against the valid 8-bit signed range.

For signed addition, overflow occurs when:

- two positive numbers produce a negative result, or
- two negative numbers produce a positive result.

Here:

120+50=170>127

Therefore:

Overflow

Optimization

Use a wider intermediate data type before storing the final result. This prevents loss of information and allows explicit overflow detection.

Testing

A	B	Expected Result	Overflow
120	50	170	Yes
40	30	70	No
-100	-50	-150	Yes
50	-20	30	No

Conclusion

The bug was caused by storing an out-of-range result in an 8-bit signed variable. Using a wider intermediate variable and checking the signed range correctly detects overflow.

Q2. Ripple Carry Adder Optimization Using CLA

1. Problem Identification

In a Ripple Carry Adder, the carry must propagate sequentially:

$C0 \rightarrow C1 \rightarrow C2 \rightarrow C3 \rightarrow C4$

This causes delay.

For an n-bit RCA:

$T_{RCA} \propto n$

The bottleneck is therefore carry propagation**.**

Buggy code:

```
#include <stdio.h>

int main()

{

   int A = 11;
```

```
    int B = 6;
    int carry = 0;
    int result = 0;
    for (int i = 0; i < 4; i++)
    {
        int a = (A >> i) & 1;
        int b = (B >> i) & 1;
        int sum = a ^ b ^ carry;
        carry = (a & b) | (a & carry) | (b & carry);
        result |= (sum << i);
    }
    printf("RCA Result = %d\n", result);
    printf("Carry = %d\n", carry);
    return 0;
}
```

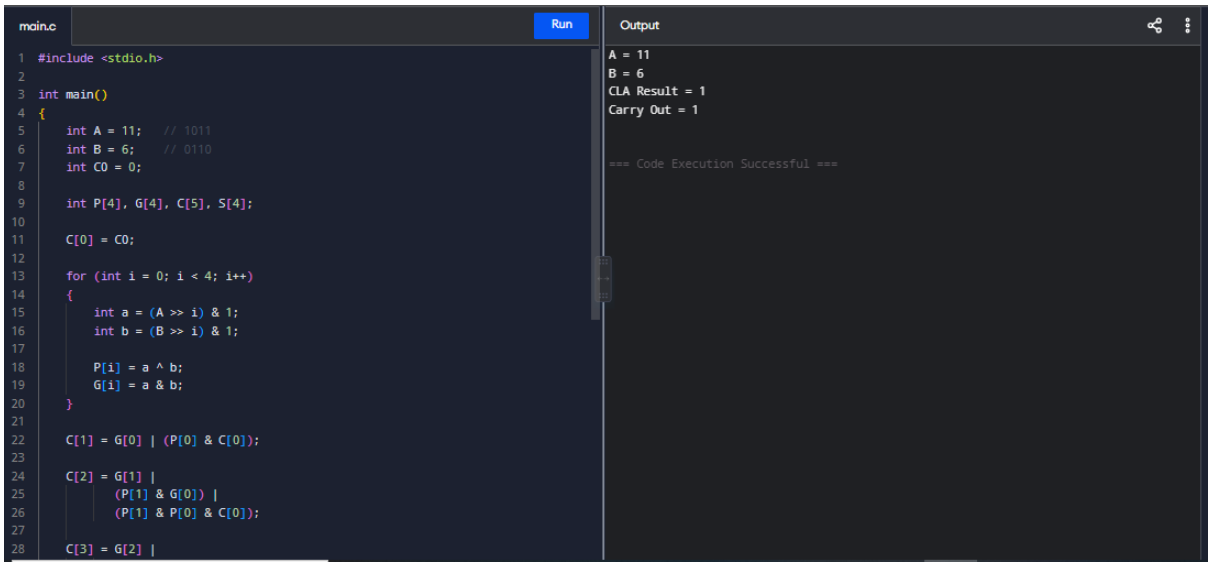
Problem

Each bit waits for the previous carry, creating sequential propagation.

Debugged Code:

```
#include <stdio.h>
int main(){
    int A = 11;   // 1011
    int B = 6;    // 0110
    int C0 = 0;
    int P[4], G[4], C[5], S[4];
    C[0] = C0;
    for (int i = 0; i < 4; i++){
        int a = (A >> i) & 1;
        int b = (B >> i) & 1;
```

```
    P[i] = a ^ b;
    G[i] = a & b;}
C[1] = G[0] | (P[0] & C[0]);
C[2] = G[1] |
       (P[1] & G[0]) |
       (P[1] & P[0] & C[0]);
C[3] = G[2] |
       (P[2] & G[1]) |
       (P[2] & P[1] & G[0]) |
       (P[2] & P[1] & P[0] & C[0]);
C[4] = G[3] |
       (P[3] & G[2]) |
       (P[3] & P[2] & G[1]) |
       (P[3] & P[2] & P[1] & G[0]) |
       (P[3] & P[2] & P[1] & P[0] & C[0]);
for (int i = 0; i < 4; i++)
    S[i] = P[i] ^ C[i];
int result = 0;
for (int i = 0; i < 4; i++)
    result |= S[i] << i;
printf("A = %d\n", A);
printf("B = %d\n", B);
printf("CLA Result = %d\n", result);
printf("Carry Out = %d\n", C[4]);
return 0;}
```



Because:

11+6=17

and:

17=100012

Therefore:

S=0001, C4=1

Debugging Analysis

The RCA waits for each carry to be generated before the next stage can proceed.

CLA calculates:

Pi=Ai⊕Bi Gi=AiBi

and generates carries directly.

Optimization

CLA reduces carry propagation delay by calculating carries in advance.

Feature	RCA	CLA
Carry	Sequential	Look-ahead
Delay	Higher	Lower
Speed	Lower	Higher
Hardware	Simple	More complex

Testing

Input: 1011 + 0110
Expected: 1 0001
Output: Carry = 1, Sum = 0001

Conclusion

The performance bottleneck is sequential carry propagation. Replacing RCA with CLA improves arithmetic speed and is suitable for high-performance processors.

Q3. Debugging Booth's Multiplication

1. Problem Identification

Booth's algorithm uses:

$Q0Q{-}1$

to determine the operation.

Q0Q−1	Operation
00	No operation
01	A=A+M
10	A=A−M
11	No operation

Common bugs include:

- Incorrect initialization of Q−1
- Checking the wrong bit pair
- Using logical instead of arithmetic right shift
- Incorrect sign extension
- Incorrect 2's complement subtraction

Buggy Code:

```
#include <stdio.h>

int main()

{

   int M = -6;

   int Q = 5;

   int A = 0;

   int Qminus1 = 0;

   for (int i = 0; i < 4; i++)

   {

      int q0 = Q & 1;

      if (q0 == 1)
```

```
        A = A + M;
    Q = Q >> 1;     // Logical shift problem for signed operation
    A = A >> 1;
  }
printf("Product = %d\n", A);
 return 0;
}
```

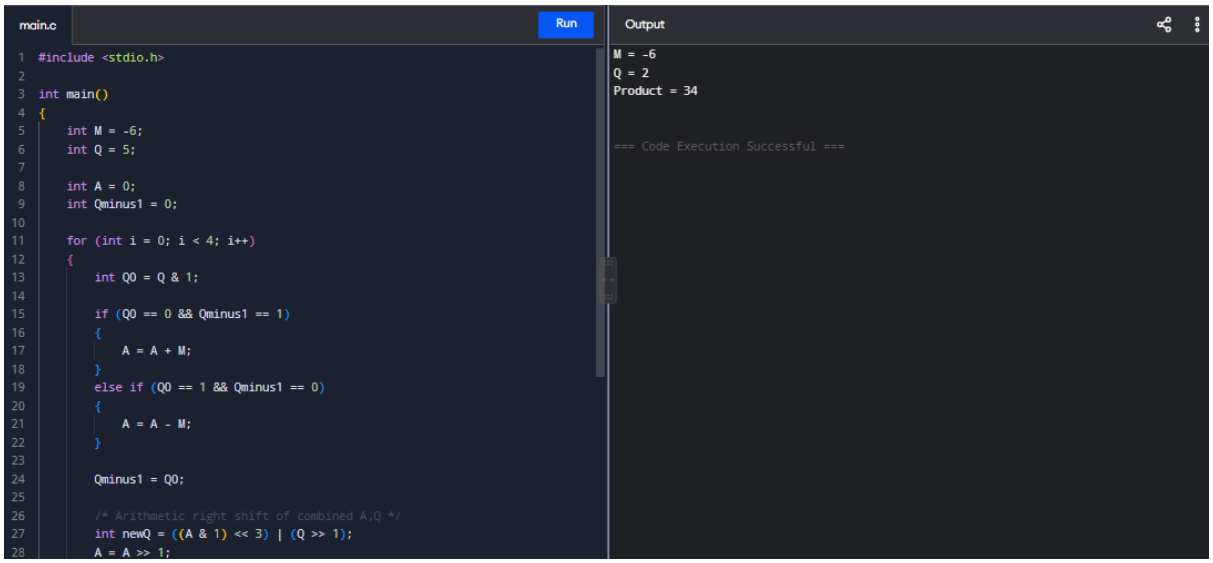
Bug

The program does not implement Booth's Q0Q−1 decision correctly and does not correctly perform the combined arithmetic shift of A,Q,Q−1.

Debugged Code

```
#include <stdio.h>
int main(){
  int M = -6;
  int Q = 5;
  int A = 0;
  int Qminus1 = 0;
  for (int i = 0; i < 4; i++){
    int Q0 = Q & 1;
    if (Q0 == 0 && Qminus1 == 1){
      A = A + M; }
    else if (Q0 == 1 && Qminus1 == 0){
      A = A - M;}
    Qminus1 = Q0;

    /* Arithmetic right shift of combined A,Q */
    int newQ = ((A & 1) << 3) | (Q >> 1);
    A = A >> 1;
    Q = newQ & 0xF;
```

```
    A = A & 0xF;}
  int product = (A << 4) | Q;
  if (product & 0x80)
    product -= 256;
  printf("M = %d\n", M);
  printf("Q = %d\n", Q);
  printf("Product = %d\n", product);
  return 0;}
```



Therefore:

(−6)×5=−30

Debugging Analysis

The correct algorithm requires:

Q0Q−1

to be examined during every iteration.

The arithmetic right shift must preserve the sign of A.

For negative numbers, sign extension must be maintained.

Optimization

Booth's algorithm reduces arithmetic operations for runs of consecutive 1s in the multiplier and naturally supports signed 2's complement multiplication.

Testing

Multiplicand	Multiplier	Expected
-6	5	-30
6	5	30
-6	-5	30
7	-3	-21

Conclusion

The primary bugs are incorrect Booth bit-pair checking and improper arithmetic shifting. Correct handling of Q0,Q−1, sign extension, and arithmetic shifts produces accurate signed multiplication.

Q4. Restoring Division Optimization Using Non-Restoring Division

1. Problem Identification

Restoring division performs:

$A=A-M$

If the result is negative, it immediately performs:

$A=A+M$

This restoration operation increases execution time.

Buggy Code

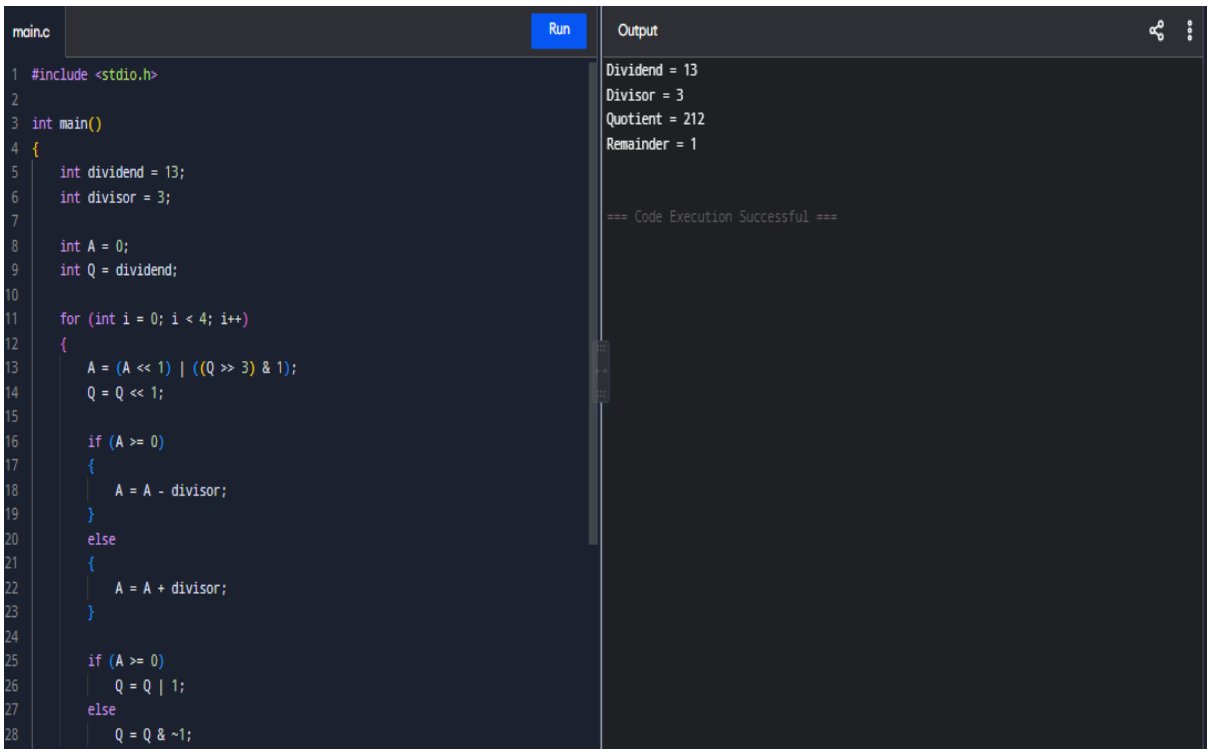
```
#include <stdio.h>

int main()

{

   int dividend = 13;

   int divisor = 3;

   int A = 0;

   int Q = dividend;

   for (int i = 0; i < 4; i++)

   {

      A = (A << 1) | ((Q >> 3) & 1);

      Q = Q << 1;
```

```
        A = A - divisor;
        if (A < 0)
        {
            A = A + divisor;   // Restoration
            Q = Q & ~1;
        }
        else
        {
            Q = Q | 1;
        }
    }
    printf("Quotient = %d\n", Q);
    printf("Remainder = %d\n", A);
    return 0;
}
```

Debugged Code:

```
#include <stdio.h>
int main(){
    int dividend = 13;
    int divisor = 3;
    int A = 0;
    int Q = dividend;
    for (int i = 0; i < 4; i++){
        A = (A << 1) | ((Q >> 3) & 1);
        Q = Q << 1;

        if (A >= 0){
            A = A - divisor;}
```

```
        else{
            A = A + divisor;}
        if (A >= 0)
            Q = Q | 1;
        else
            Q = Q & ~1;}
    if (A < 0)
        A = A + divisor;
    printf("Dividend = %d\n", dividend);
    printf("Divisor = %d\n", divisor);
    printf("Quotient = %d\n", Q);
    printf("Remainder = %d\n", A);
    return 0;}
```



Therefore:

13=3(4)+1

Debugging Analysis

Restoring division immediately restores a negative partial remainder.

Non-restoring division instead uses the sign of the previous partial remainder to decide whether to add or subtract the divisor in the next iteration.

Optimization

Feature	Restoring	Non-Restoring
Restoration	Frequent	Avoided
Operations	More	Fewer
Execution time	Higher	Lower
Efficiency	Lower	Higher

Testing

Dividend	Divisor	Quotient	Remainder
13	3	4	1
15	3	5	0
10	2	5	0
14	4	3	2

Conclusion

The repeated restoration operation is the main inefficiency. Non-restoring division avoids unnecessary restoration and improves arithmetic-unit performance.

Q5. Debugging IEEE 754 Floating-Point Addition

Problem Identification

IEEE 754 single precision consists of:

1 sign bit+8 exponent bits+23 fraction bits

The problem occurs when numbers have a large difference in exponent.

Example:

A=1.000000×220 B=1.000000×2−10

Exponent difference:

20−(−10)=30

The smaller number's mantissa must be shifted right by 30 positions. Since single precision has limited significant bits, information is lost.

Buggy Code

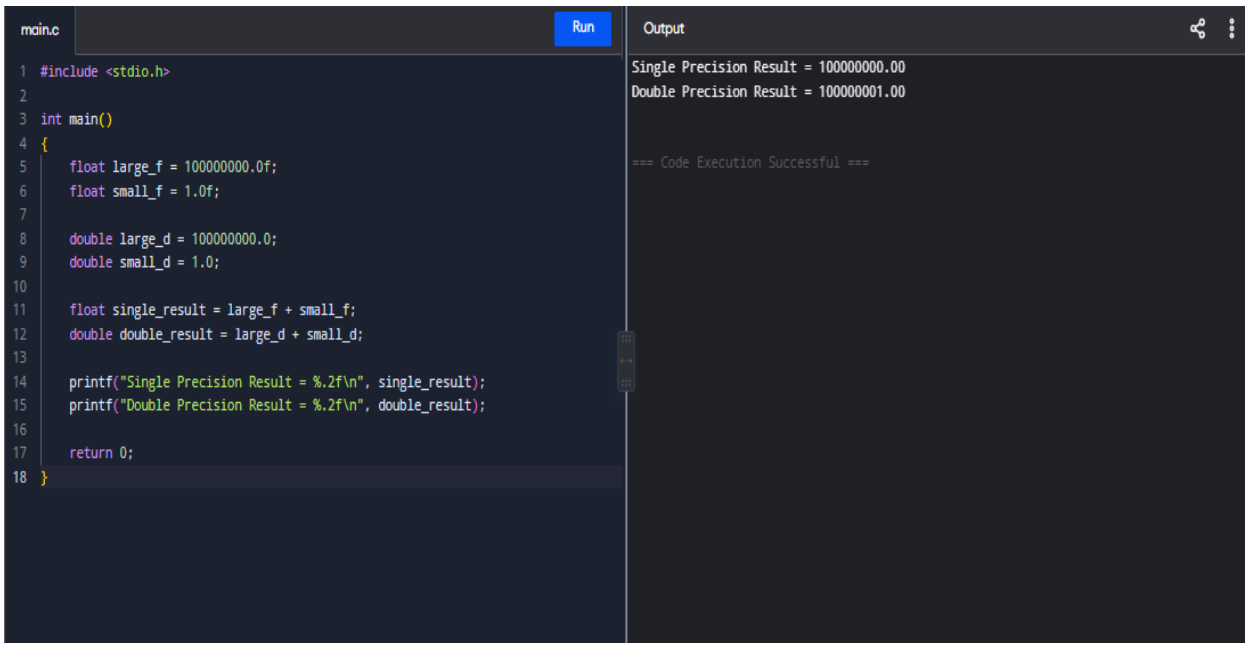
```
#include <stdio.h>

int main()
{
   float large = 100000000.0f;
   float small = 1.0f;

   float result = large + small;

   printf("Result = %.2f\n", result);

   return 0;
}
```

Problem

Single precision has limited precision. When a very small value is added to a much larger value, the small value may not affect the stored result.

Debugded Code:

```
#include <stdio.h>

int main()

{

    float large_f = 100000000.0f;

    float small_f = 1.0f;

    double large_d = 100000000.0;

    double small_d = 1.0;

    float single_result = large_f + small_f;

    double double_result = large_d + small_d;

    printf("Single Precision Result = %.2f\n", single_result);

    printf("Double Precision Result = %.2f\n", double_result);

    return 0;

}
```



Debugging Analysis

Floating-point addition follows these steps:

Step 1: Compare Exponents
E1, E2

are compared.

Step 2: Align Exponents

The mantissa corresponding to the smaller exponent is shifted right.

Step 3: Add Mantissas

MR=M1+M2

Step 4: Normalize

The result is converted to:

1.xxxxx×2E

Step 5: Round

Excess fraction bits are rounded according to IEEE 754 rounding rules.

Sources of Error

- Exponent alignment
- Limited fraction bits
- Rounding
- Cancellation
- Repeated arithmetic operations

Optimization Techniques

1. Use Double Precision

Single precision:

23 fraction bits

Double precision:

52 fraction bits

Therefore, double precision provides significantly greater accuracy.

2. Reduce Repeated Rounding

Perform intermediate calculations using higher precision before converting to lower precision.

3. Compensated Summation

For repeated addition, compensated algorithms can reduce accumulated floating-point error.

4. Group Similar-Magnitude Values

Adding values with similar magnitudes reduces the amount of significant information lost during exponent alignment.

Testing

Large Value	Small Value	Single Precision	Double Precision
108	1	May lose 1	Preserved
108	10	Limited effect	Better
104	1	Accurate	Accurate
104	-1	Accurate	Accurate

Conclusion

The inaccurate result is caused by limited precision during exponent alignment and rounding**.** Using double precision, higher-precision intermediate calculations, and suitable numerical algorithms improves accuracy.